Minimizing DFA's

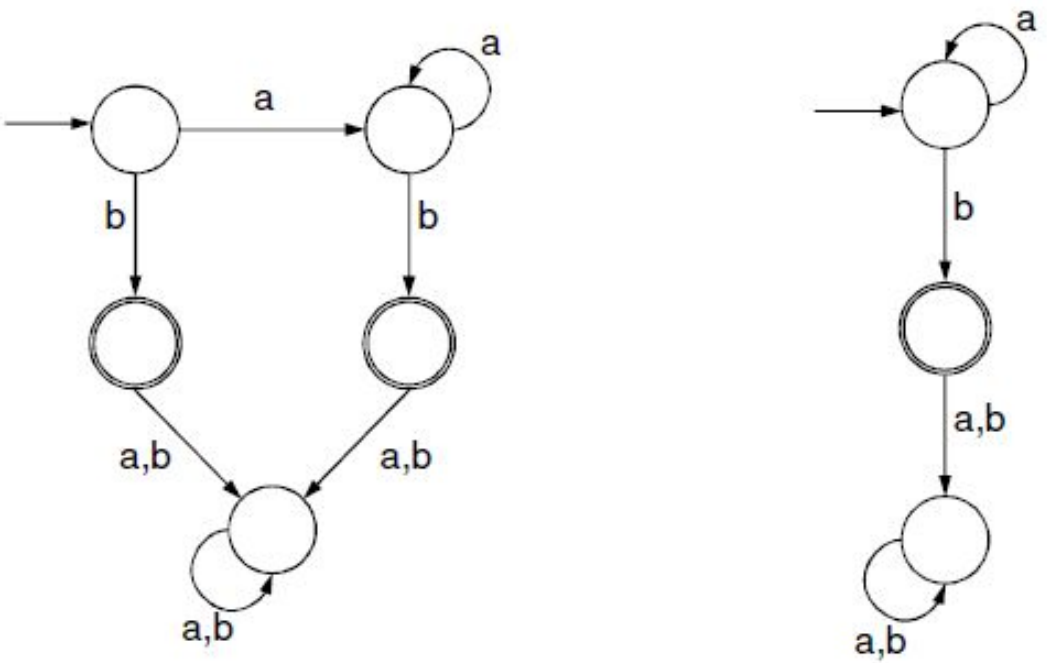
By Partitioning

Why do we want to minimize a DFA?

- Efficiency
- Better understanding on the language recognized by the DFA
- Computing the similarity between two Fas
- Determining if two DFAs recognize the same language

Two DFAs

- Do these two DFAs accept the same language?



Minimization Process

- Get rid of inaccessible states
- Group equivalent states
  - Two states are equivalent if all subsequent behavior from these states is the same.

DFA Minimization

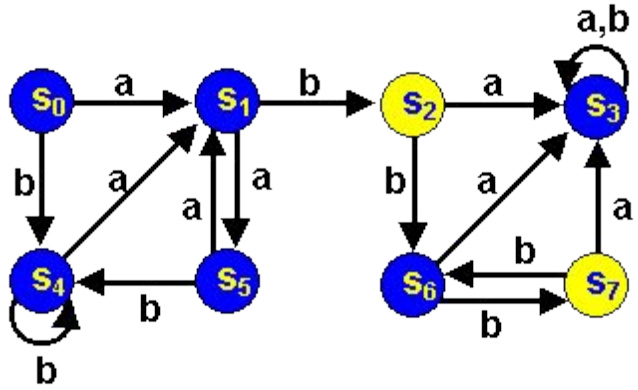
1. Place the states of the DFA into two equivalence classes: final states and nonfinal states.
2. Repeat this step until there is no more change i.e., no new equivalence classes:
   - Determine the transitions with respect to equivalence classes.
   - For each state in the same equivalence class, if the transition is different than those in its group, partition it to a new equivalence class.

Minimizing DFA's

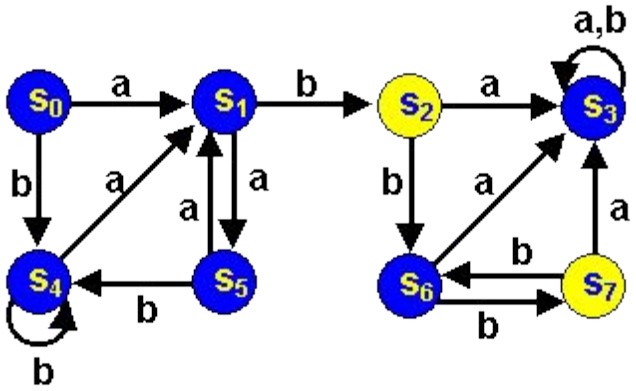
- ***Lots*** of methods
- All involve finding **equivalent states**:
  - States that go to equivalent states under all inputs (sounds recursive)
- We will use the ***Partitioning Method***

Minimizing DFA's by Partitioning

Consider the following dfa (from Forbes Louis at U of KY):



- **Accepting** states are **yellow**
- Non-accepting states are **blue**
- Are any states really the same?



- S_2 and S_7 are really the same:
  - Both Final states
  - Both go to S6 under input *b*
  - Both go to S3 under an *a*
- S_0 and S_5 really the same. Why?
- We say each pair is equivalent

Are there any other equivalent states?
We can merge equivalent states into 1 state

Partitioning Algorithm

- First
  - Divide the set of states into

    Final and
    Non-final states

Partition I

Partition II

	a	*b*
S_0	S_1	S_4
S_1	S_5	S_2
S_3	S_3	S_3
S_4	S_1	S_4
S_5	S_1	S_4
S_6	S_3	S_7
$*S_2$	S_3	S_6
$*S_7$	S_3	S_6

Partitioning Algorithm

- Now
  - See if states in each partition each go to the same partition
- S_1 & S_6 are different from the rest of the states in Partition I (but like each other)
- We will move them to their own partition

	a	*b*
S_0	S_1 I	S_4 I
S_1	S_5 I	S_2 II
S_3	S_3 I	S_3 I
S_4	S_1 I	S_4 I
S_5	S_1 I	S_4 I
S_6	S_3 I	S_7 II
*S_2	S_3 I	S_6 I
*S_7	S_3 I	S_6 I

Partitioning Algorithm

	a	*b*
S_0	S_1	S_4
S_5	S_1	S_4
S_3	S_3	S_3
S_4	S_1	S_4
S_1	S_5	S_2
S_6	S_3	S_7
$*S_2$	S_3	S_6
$*S_7$	S_3	S_6

Partitioning Algorithm

- Now again
  - See if states in each partition each go to the same partition
  - In Partition I, S_3 goes to a different partition from S_0, S_5 and S_4
  - **We'll move S3 to its own partition**

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_3	S_3 I	S_3 I
S_4	S_1 III	S_4 I
S_1	S_5 I	S_2 II
S_6	S_3 I	S_7 II
$*S_2$	S_3 I	S_6 III
$*S_7$	S_3 I	S_6 III

Partitioning Algorithm

Note changes in S_6, S_2 and S_7

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_4	S_1 III	S_4 I
S_3	S_3 IV	S_3 IV
S_1	S_5 I	S_2 II
S_6	S_3 IV	S_7 II
$*S_2$	S_3 IV	S_6 III
$*S_7$	S_3 IV	S_6 III

Partitioning Algorithm

- Now S_6 goes to a different partition on an *a* from S_1
- S_6 gets its own partition.
- We now have 5 partitions
- Note changes in S_2 and S_7

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_4	S_1 III	S_4 I
S_3	S_3 IV	S_3 IV
S_1	S_5 I	S_2 II
S_6	S_3 IV	S_7 II
$*S_2$	S_3 IV	S_6 V
$*S_7$	S_3 IV	S_6 V

Partitioning Algorithm

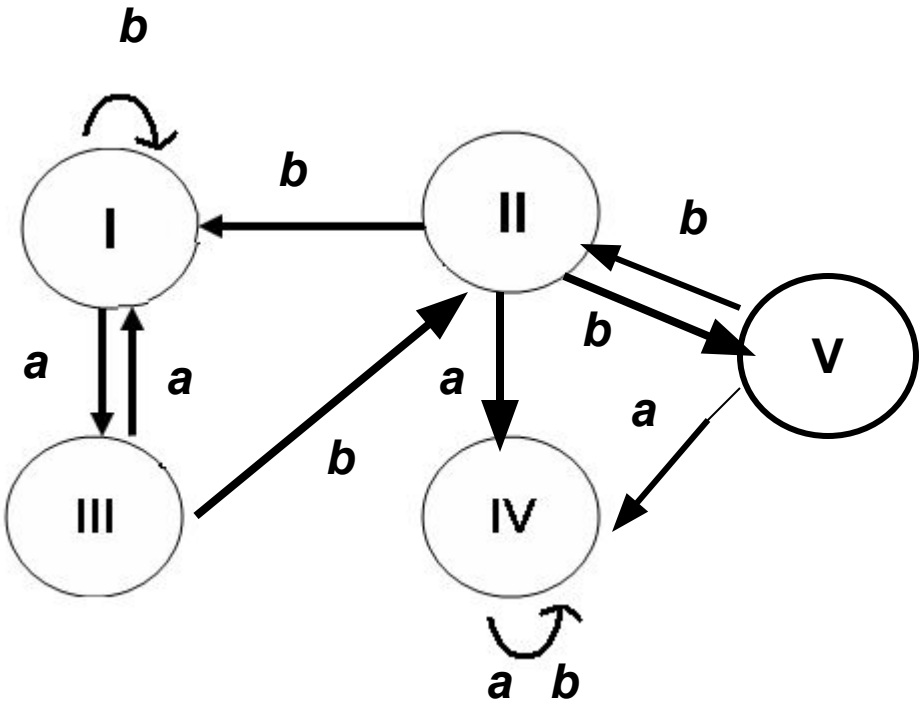
- All states within each of the 5 partitions are identical.
- We might as well call the states I, II III, IV and V.

	a	*b*
S_0	S_1 III	S_4 I
S_5	S_1 III	S_4 I
S_4	S_1 III	S_4 I
S_3	S_3 IV	S_3 IV
S_1	S_5 I	S_2 II
S_6	S_3 IV	S_7 II
$*S_2$	S_3 IV	S_6 V
$*S_7$	S_3 IV	S_6 V

Partitioning Algorithm

Here they are:

	a	*b*
I	III	I
*II	IV	V
III	I	II
IV	IV	IV
V	IV	II


b
I
b
II
b
V
b
a
a
a
a
b
III
IV
a
b

DFA Minimization using Myphill-Nerode Theorem (TF Algorithm)

Algorithm

Input – DFA

Output – Minimized DFA

Step 1 – Draw a table for all pairs of states (Q_i, Q_j) not necessarily connected directly [All are unmarked initially]

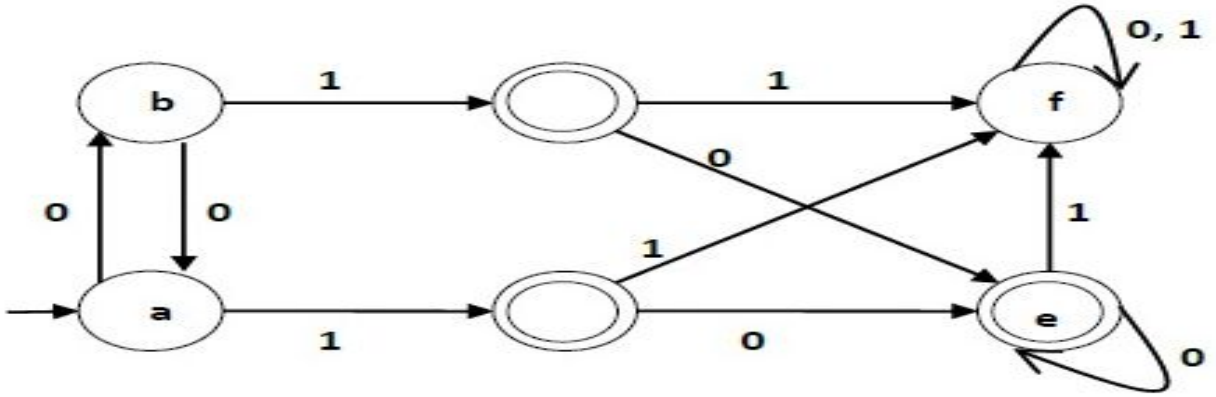
Step 2 – Consider every state pair (Q_i, Q_j) in the DFA where $Q_i \in F$ and $Q_j \notin F$ or vice versa and mark them. [Here F is the set of final states]

Step 3 – Repeat this step until we cannot mark anymore states –

If there is an unmarked pair (Q_i, Q_j), mark it if the pair $\{\delta (Q_i, A), \delta (Q_i, A)\}$ is marked for some input alphabet.

Step 4 – Combine all the unmarked pair (Q_i, Q_j) and make them a single state in the reduced DFA.

Example



Step 1 – We draw a table for all pair of states.

	a	b	c	d	e	f
a						
b						
c						
d						
e						
f						

Step 2 – We mark the state pairs.

	a	b	c	d	e	f
a						
b						
c	✓	✓				
d	✓	✓				
e	✓	✓				
f			✓	✓	✓	

Step 3 – We will try to mark the state pairs, with green colored check mark, transitively. If we input 1 to state 'a' and 'f', it will go to state 'c' and 'f' respectively. (c, f) is already marked, hence we will mark pair (a, f). Now, we input 1 to state 'b' and 'f'; it will go to state 'd' and 'f' respectively. (d, f) is already marked, hence we will mark pair (b, f).

	a	b	c	d	e	f
a						
b						
c	✓	✓				
d	✓	✓				
e	✓	✓				
f	✓	✓	✓	✓	✓	

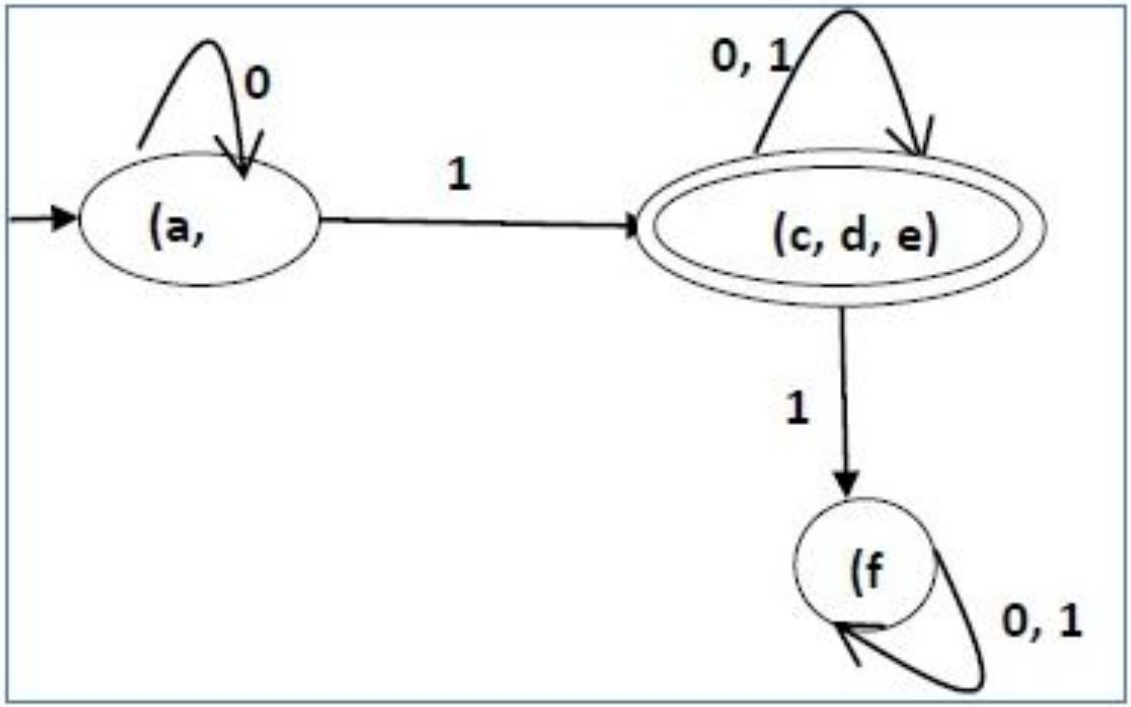
After step 3, we have got state combinations {a, b} {c, d} {c, e} {d, e} that are unmarked.

We can recombine {c, d} {c, e} {d, e} into {c, d, e}

Hence we got two combined states as – {a, b} and {c, d, e}

So the final minimized DFA will contain three states {f}, {a, b} and {c, d, e}

Minimized DFA



Minimized DFA Cannot be Beaten

Let B be the minimized DFA obtained by applying the TF-Algo to DFA A. We know that L(A) = L(B)

What if there existed a DFA C with L(C) = L(B) and fewer states than B?

If we run the TF-Algo on B "union" C

$$q_0^B \equiv q_0^C \Rightarrow L(B) = L(C)$$

Also $\delta(q_0^B, a) \equiv \delta(q_0^C, a), \forall a \in \sum$

If we could distinguish successors then $q_0^B \not\equiv q_0^C$

Minimized DFA Cannot be Beaten (cont)

Claim: For each state p in B there is at least one state q in C, s.t. $p \equiv q$

Proof: There are no inaccessible states, so

$p = \delta^*(q_0^B, a_1 a_2 \cdots a_k)$

Now $q = \delta^*(q_0^C, a_1 a_2 \cdots a_k)$ and $p \equiv q$

$\because q_0^B \equiv q_0^C \therefore \delta(q_0^B, a_1) \equiv \delta(q_0^C, a_1)$

$\delta^*(q_0^B, a_1 a_2) \equiv \delta^*(q_0^C, a_1 a_2)$

Since C has fewer states than B, there must be two states r & s of B s.t. $r \equiv t \equiv s$ For some state t of C. But $r \equiv s$ CONTRADICTION!!